PRACTICAL 9

Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing

AIM

To implement secure user registration in StudentHub using PHP and MySQLi, validate input on the frontend and backend, check duplicate emails, hash passwords using password_hash(), and display appropriate messages.

Introduction

StudentHub was extended to store user accounts in studenthub_db. Registration details are validated before inserting them using a MySQLi prepared statement. Duplicate emails are rejected, and passwords are stored as hashes. Tests confirmed successful registration, invalid-input handling and duplicate-email rejection.

Tools and Technology

VS Code, XAMPP, MariaDB, phpMyAdmin, PHP, MySQLi, HTML, CSS, JavaScript, SQL, browser, Git and GitHub. MariaDB is the MySQL-compatible database server supplied with XAMPP.

Files Used

File	Purpose
register.html	User registration form and field error messages.
js/validation.js	Frontend validation and registration submission.
php/process-registration.php	Backend validation, duplicate check, hashing and insert.
php/db-mysqli.php	Reusable MySQLi database connection.
php/db-config.php	Private database settings; excluded from Git.
php/get-csrf-token.php	Session CSRF token and security code.
sql/practical9.sql	Users and registration audit table definitions.

Key Skills Used

Secure registration, frontend and backend validation, MySQLi connectivity, prepared statements, duplicate checks, password hashing, database inserts, CSRF protection and audit logging.

Implementation Overview

1 Database and Tables

The users table stores full name, email, mobile, password hash, course, year, gender, role and creation time. user_id is an AUTO_INCREMENT primary key, and email has a UNIQUE constraint. The registration_audit table records successful account creation.

2 Frontend and Backend Validation

JavaScript validates the form before submission. PHP repeats the validation on the server. Both apply matching rules for name, email, mobile, password and selection fields. Password confirmation and acceptance of terms are required.

3 MySQLi Connectivity

db-mysqli.php reads the local database configuration and connects using the studenthub_app account. The connection uses utf8mb4 and MySQLi exception reporting. The private configuration file remains excluded from Git.

4 Duplicate Email Check

A prepared SELECT checks whether the email already exists. The email is trimmed and converted to lowercase. A UNIQUE constraint prevents duplicate inserts, and duplicate requests receive an appropriate email error.

5 Password Hashing

The password is hashed using password_hash() with PASSWORD_DEFAULT before insertion. The password_hash column uses VARCHAR(255). Plaintext passwords are not stored. password_verify() confirmed the correct test password and rejected an incorrect password.

6 Prepared Insert and Error Handling

User details are bound as parameters to a prepared INSERT statement. Valid input creates an account and displays a success message. Validation errors are returned against their fields. Database errors return a general message, with details logged on the server.

7 CSRF Token and Audit Log

A session CSRF token is generated and checked using hash_equals(). Incorrect tokens are rejected. Account creation and its audit entry are saved in one transaction, so both operations commit together.

Screenshots of Registration and Validation Results

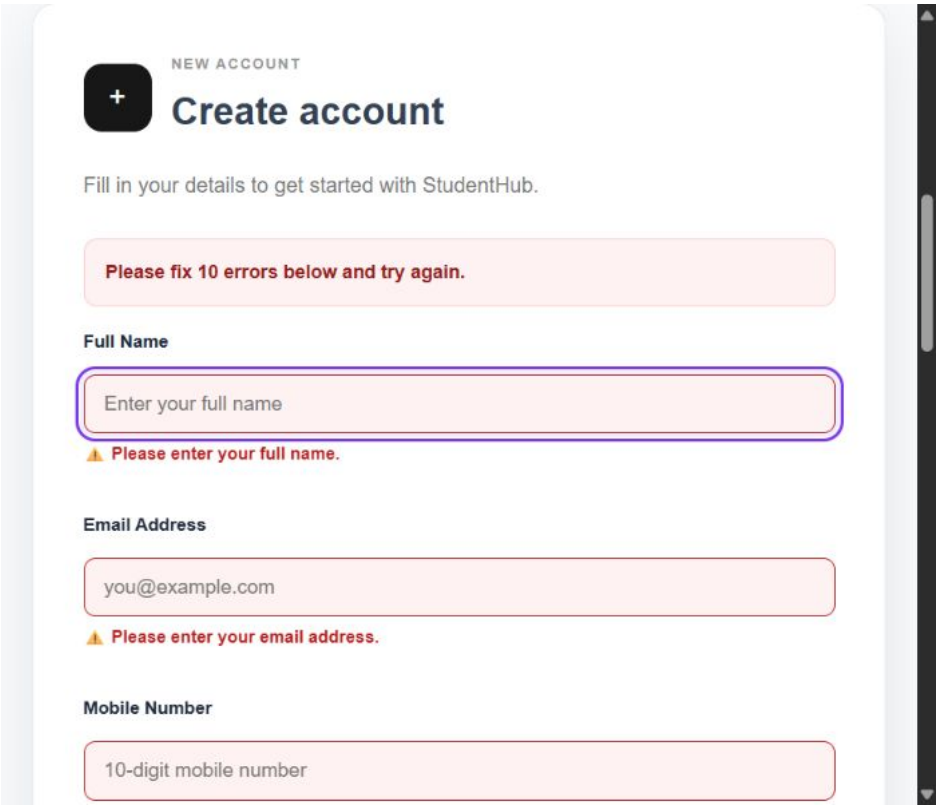


Figure 1: Registration form displaying required-field validation errors.



Figure 2: Recorded HTTP responses for valid, duplicate and invalid email registration.

Screenshots of Database Insert and Audit Log

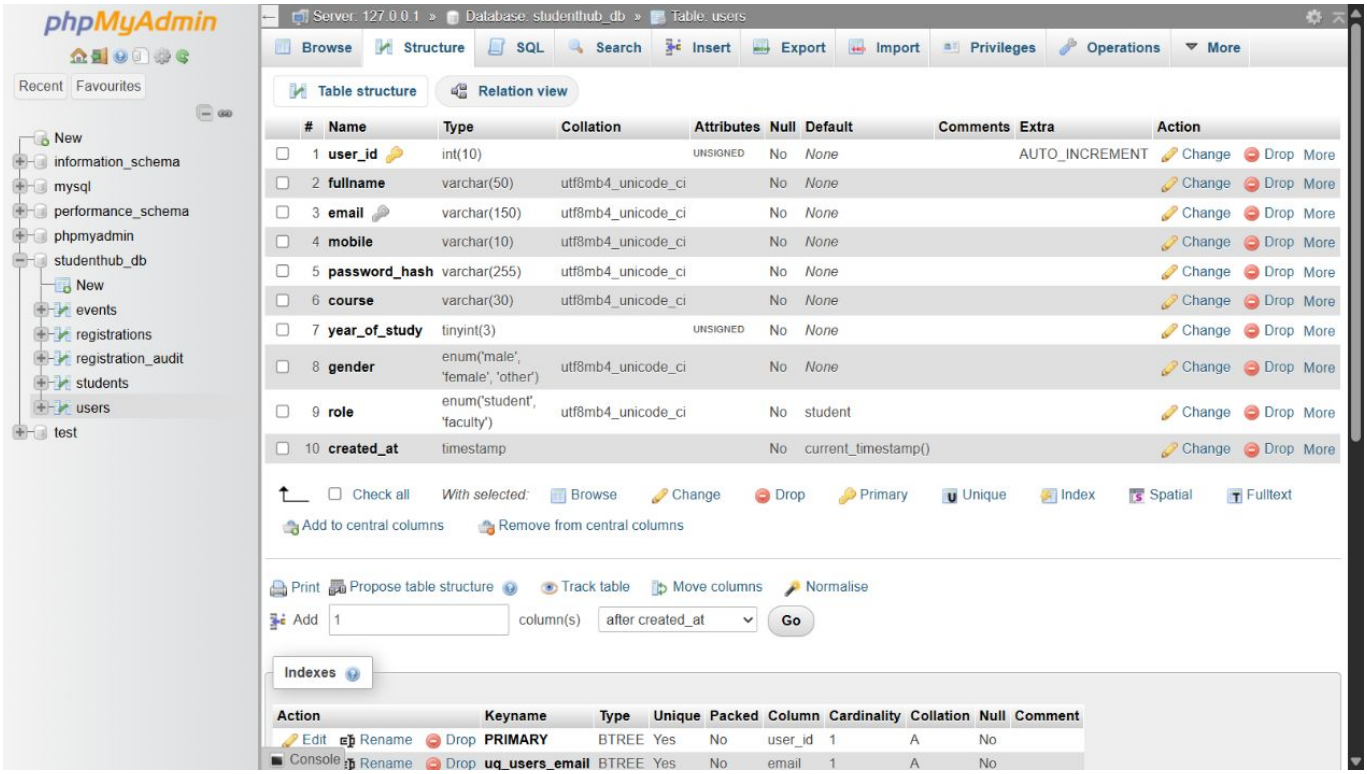


Figure 3: phpMyAdmin users table structure showing the password_hash column.

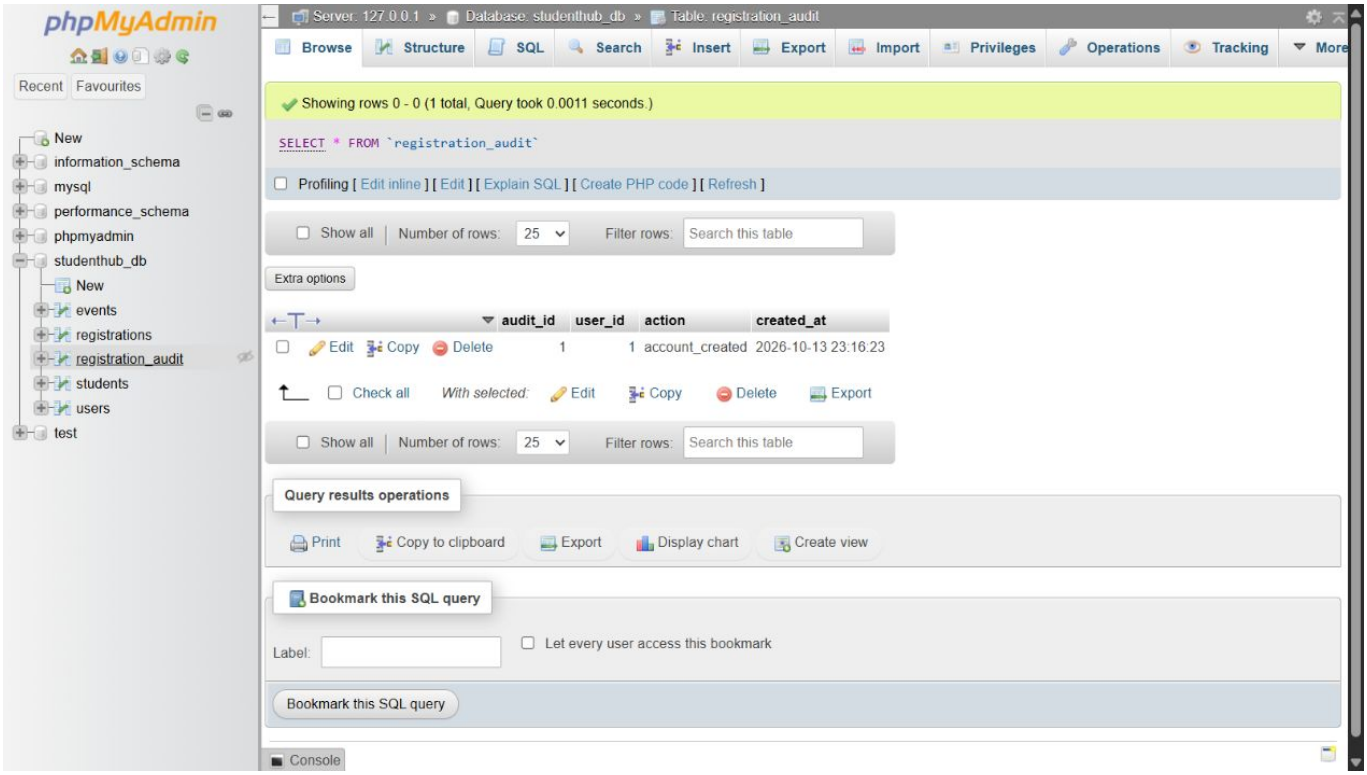


Figure 4: Successful account_created entry in registration_audit.

Database Structure and Registration Results

Table	Primary key	Other constraints
users	user_id	Unique email; year CHECK 1-4
registration_audit	audit_id	Foreign key referencing users.user_id

Valid Registration

Full name: Practical Test
Email: practical9.1791913582@example.com
Course: BCA; year: 1
Result: Welcome, Practical Test! Your StudentHub account has been created.

Database Verification

Check	Observed result
Inserted account	User ID 1; Practical Test
Password storage	Hash stored instead of plaintext
Correct password	Verified successfully
Incorrect password	Rejected
Audit log	One account_created entry

Testing Summary

Test input or action	Observed result
Valid registration	Account created successfully.
Duplicate email / changed email case	Rejected: this email is already registered.
Invalid email	Rejected with an email validation error.
Weak password / mismatched confirmation	Rejected with password errors.
Invalid course, year or role	Rejected with field validation errors.
Terms not accepted	Rejected: terms must be accepted.
Incorrect security code	Rejected with a security-code error.
Incorrect CSRF token	Rejected with a session error.

Registration URL: http://localhost/studenthub-portal/register.html

Key Questions and Answers

Q1 Is password_hash() used before storing passwords

Yes. PHP calls password_hash() with PASSWORD_DEFAULT before inserting the account. Only the resulting hash is stored in password_hash. The stored hash was verified using password_verify().

Q2 Is duplicate email/username checked

Email is used as the unique identifier in the StudentHub registration form. A prepared query checks for an existing email, and a UNIQUE constraint prevents duplicate inserts. Tests rejected identical emails and duplicates with different capitalization.

Q3 Are frontend and backend validations consistent

JavaScript and PHP apply matching rules for names, emails, mobile numbers, passwords and allowed field values. PHP also verifies the CSRF token and security code. Invalid-input tests confirmed that backend validation rejects unsuitable data.

Q4 Why use a prepared statement

Prepared statements keep SQL structure separate from supplied values. User input is bound as parameters rather than concatenated into the query, protecting the insert and duplicate-email query from SQL injection.

GitHub Repository

https://github.com/DarshilRupani/studenthub-portal
Practical 9 source commit: 6a89671
Message: Complete Practical 9 secure MySQLi user registration

Post Laboratory Work

Submit the registration source files, sql/practical9.sql and this registration test report covering valid, invalid and duplicate cases. Start Apache and MySQL, import the new table definitions into studenthub_db, and use the local database configuration to reproduce the setup.

Conclusion

Secure user registration was implemented using MySQLi. Valid details were inserted, duplicate emails were rejected, and passwords were stored as hashes. Validation and password verification tests passed. The intermediate prepared-statement extension and the advanced CSRF token and audit log extension were completed.

Verification scope: HTTP and database tests confirm the recorded results. Required-field validation was also checked in the browser. Concurrent duplicate requests and a forced audit-write failure were not tested.